JULIANA AMPARO CABRERA

CatBoost Neural Learning

Test JAC

Document Details

Submission ID

trn:oid:::3618:120262601

Submission Date

Nov 8, 2025, 8:00 AM GMT+8

Download Date

Nov 8, 2025, 8:02 AM GMT+8

File Name

CatBoost Neural Learning.docx

File Size

43.4 KB

7 Pages

1,685 Words

9,428 Characters

Feedback Summary

Final Grade

undefined/100

Feature Engineering Discussion

Date:

Think of the regression model like a smart calculator that ONLY UNDERSTANDS numbers.

From the dataset given, we got a column for dates that our so called “Calculator capability” wouldn’t regard as a number. It will for sure spit out an “error” result.

However, the sales are typically dependent on when the date is let’s say for example:

Sales may go up in December (especially for Christmas time)

Sales might change year-over-year (2010, 11, or 12)

Sales might be lower on the first week of the month, than the last week of a recent month

Re-Engineering Method Applied:

Re-engineering the date (predictor variable) feature, it’ll be expanded to produce 3 new column features splitting them into: Year, Month, and the Week Number.

With that kind of approach, it will help aid the regression model/s to say things and execute the principle of “For every one-unit increase in a given month (a number for example going from October to November), the weekly sales (Dependent variable) will go up by xx,xxx.00”, which makes the model predictive capable.

Store:

Bear in mind that the data value of this feature is NOT a number, it is a **Label** or an **Identifier** of a Physical Store Location. Therefore, it is regarded as a Nominal/Categorical Data.

Let’s say Store column with a value of 4 **IS NOT** as twice a good as Store column with a value of 2. Or Store 10 is not 5 units better than Store 5.

Whenever we took the value as a number “x”, the model will believe that the higher numbers means higher sales which is NOT TRUE. Sales are high or low because of the store’s location, not the arbitrary ID number.

Re-engineering Method applied (for CatBoost model):

This is the only feature that needs to be broken down into simpler, more useful components to make it useable for the model

Illustrating the re-engineered concept for this feature, this how we re-label the stores, let's use this Date value from Dataset: **5/2/2010**

New Feature	Description	Value
Day of Year	Day number within the year (1-366)	122
Week of Year	Week number within the year (1-53)	17
Month	Month Number (1-12)	5
Year	Year Number (2010, 2011, etc)	2010

By using these newly- generated (expanded) columns, the model will be navigated to treat each store as a separate, unique location. This will feed the decision statement of "When we have this store 36 flag, add $ xx,xxx.00 to the predicted sales".

<u>Holiday_Flag:</u>

It's generally recognized that a column that has only two (2) options like True or False, Yes or No, and 0 or 1 is called a "Binary Variable", therefore, it fits the requirement.

<u>Temperature / Fuel_Price / CPI / Unemployment:</u>

These independent variables are Continuous types (numbers with decimals), represent real, measurable quantities. They are already in a perfected state for our model/s to accept.

To Summarize things in a tabulated way:

Column	Type	Action Taken	Re-Engineered Results
Weekly_Sales	Continuous	Target (Dependent Variable)	This is to be predicted
Date	Date/Time	Feature Engineering	Year, Month, Week
Stores	Categorical	One- Hot Encoding	One unique col per store
Holiday_Flag	Binary (0/1)	No Change	Use As-Is
Temperature, Fuel_Price, CPI, Unemployment	Continuous	No Change	Use As-Is

Neural Learning Discussion of CatBoost model in Weekly Sales Prediction

After all the Feature Engineering processes and realized all the data characteristics, let’s deep dive and understand this model’s works in a step-by-step way.

Ordered Target Encoding Discussion

What makes this model differ from other gradient boosting algorithms is that it automatically handles categorical feature (i.e. on our Store), using this technique to prevent any instances of “Prediction Shift”.

Let’s say if whenever we calculate the average Weekly Sales for **Store 1** and use that number as the encoding value, the model would be “leaking” information from the target variable (Weekly Sales) into the feature (Store), leading to “overly optimistic” results.

To prevent this problem, the model performs a “slow and sequential” process. It uses a random “shuffle” of the dataset and calculates the encoding for a given row based only on the data before it in the shuffled lists. Let’s put this into action..

We took 8 row items of Stores (1 and 2) to explain the concept, we can see how the model converts their categorical values into chunks (encoded feature values) that will be used in training trees to predict the target variable.

Before Random Shuffling:

Row ID	Store	Weekly Sales
1	1	1643691
2	1	1641957
3	1	1611968
4	1	1409728
5	1	1554807
6	1	1439542
7	1	1472516
8	1	1404430
9	2	2136989
10	2	2137810
11	2	2124452
12	2	1865097
13	2	1991013
14	2	1990484
15	2	1946071
16	2	1750198

The model's algorithm doing random shuffling of rows:

[1,2,6,15,14,13,4,9,7,10,5,16,8,11,12,3] → These are the re-arranged row numbers of the dataset

Formulas/Equations to be used:

GlobalAverage = $(\sum Sales) \,/\, no.of\ items$ = 1757547.02

Encoded Feature (for Stores) =

((sum of sales for same category)+G.A.)*a**)** / **(**count of category before * a**)**

*Where **a** is considered a hyperparameter. Its optimal value is not fixed and must be tuned relative to the dataset's characteristics. The default value of 1 is usually a safe starting point that offers reasonable balance between local and global information.*

After Random Shuffling:

Orig Row ID	Store	Sales	Category Previous	Sum Y (before)	COCB	Formulas	Resulting Encoded Features
1	1	1643691	0	0	0	(0+1757547.02)/(0+1)	1757547
2	1	1641957	1	1643691	1	(1643691+1757547.02)/(1+1)	1700619
6	1	1439542	2	3285648	2	(1643691+1641957+1757547)/(2+1)	1681065
15	2	1946071	0	0	3	(0+1757547.02)/(0+1)	1757547
14	2	1990484	1	1946071	4	(1946071+1757547.02)/(1+1)	1851809
13	2	1991013	2	3936555	5	(3936555+1757547.02)/(2+1)	1898034
4	1	1409728	3	4695376	6	(4695376+1757547)/(3+1)	1613231
9	2	2136989	3	5927568	7	(5927568+1757547)/(3+1)	1921279
7	1	1472516	4	6167892	8	(6167892+1757547)/(4+1)	1585088
10	2	2137810	4	8064557	9	(8064557+1757547)/(4+1)	1964421
5	1	1554807	5	7722699	10	(7722699+1757547)/(5+1)	1580041
16	2	1750198	5	10202367	11	(10202367+1757547)/(5+1)	1993319
8	1	1404430	6	9127129	12	(9127129+1757547)/(6+1)	1554954
11	2	2124452	6	11952565	13	(11952565+1757547.02)/(6+1)	1958587
12	2	1865097	7	14077016	14	(14077016+1757547.02)/(7+1)	1979320
3	1	1611968	7	10739097	15	(10739097+1757547.02)/(7+1)	1562081

*** COCB means "Count of Category Before"*

The Resulting Encoded Features above is now the new "**unique**" numerical representation of the Store column, ready to be used by the first decision tree (T_1).

Gradient Calculation Discussion

This is vital to have so that the model's algorithm can realize exactly what are the "mistakes" that needs to be fixed, ensuring that the next decision tree/s to be trained effectively contribute to reduce the error of the previous decisions and create a strong prediction.

Formulas/Equations to be used:

$\text{Residual1}_{(\text{TargetTree1})}$ = Actual Weekly Sales – *Initial Prediction*

where Initial Prediction == GlobalAverage

$\text{Residual2}_{(\text{TargetTree2})}$

= Actual Weekly Sales – (*Initial Prediction* + n + $\text{Residual1}_{(\text{TargetTree1})}$)

where n is the learning rate

Store	Orig Row ID	Sales	Initial Prediction	R_1 Calculation (Sales-Ini. Prediction)	$\text{Residual1}_{(\text{TargetTree1})}$
1	1	1643691	1757547.02	1643691 - 1757547.02	-113856.02
1	2	1641957	1757547.02	1641957- 1757547.02	-115590.02
1	6	1439542	1757547.02	1439542 - 1757547.02	-318005.02
2	15	1946071	1757547.02	1946071 - 1757547.02	188523.98
2	14	1990484	1757547.02	1990484 - 1757547.02	232936.98
2	13	1991013	1757547.02	1991013 - 1757547.02	233465.98
1	4	1409728	1757547.02	1409728 - 1757547.02	-347819.02
2	9	2136989	1757547.02	2136989 - 1757547.02	379441.98
1	7	1472516	1757547.02	1472516 - 1757547.02	-285031.02
2	10	2137810	1757547.02	2137810 - 1757547.02	380262.98
1	5	1554807	1757547.02	1554807 - 1757547.02	-202740.02
2	16	1750198	1757547.02	1750198 - 1757547.02	-7349.02
1	8	1404430	1757547.02	1404430 - 1757547.02	-353117.02
2	11	2124452	1757547.02	2124452 - 1757547.02	366904.98
2	12	1865097	1757547.02	1865097 - 1757547.02	107549.98
1	3	1611968	1757547.02	1611968 - 1757547.02	-145579.02

From the results, Store 1 (highlighted) dominates the **Negative** residuals, on the other hand, Store 2 for **Positive** residuals.

Depth of the First Decision Tree (F_1)

The goal is to find the best split (from 16 rows), reducing the error as much as possible. Split point (basis of split) is ≤ **1,757,547.02.**

LEFT LEAF			
ROW No.	CALCULATION (R1=Y-f1)	Residual(R1)	R^2
01	1643691 – 1757547.02	-113856.02	12963000000
02	1641957 – 1757547.02	-115590.02	13361000000
03	1611968 – 1757547.02	-145579.02	21192000000
04	1409728 – 1757547.02	-347819.02	120978000000
05	1554807 – 1757547.02	-202740.02	41103000000
06	1439542 – 1757547.02	-318005.02	101127000000
07	1472516 – 1757547.02	-285031.02	81243000000
08	1404430 – 1757547.02	-353117.02	124692000000
TOTAL		**-1911726.16**	516658000000

RIGHT LEAF			
ROW No.	CALCULATION (R1=Y-f1)	Residual(R1)	R^2
9	2136989.46 – 1757547.02	379442.44	143976000000
10	2137809.50 – 1757547.02	380262.48	144599000000
11	2124451.54 – 1757547.02	366904.52	134608000000
12	1865097.27 – 1757547.02	107550.25	11567000000
13	1991013.13 – 1757547.02	233466.11	54517000000
14	1990483.78 – 1757547.02	232936.76	54233000000
15	1946070.88 – 1757547.02	188523.86	35541000000
16	1750197.81 – 1757547.02	-7349.21	54000000
TOTAL		**1885081.33**	**579195000000**

From the resulting table on both leaves, we now compute the predictions for every leaf:

$\text{Prediction}_{\textbf{(Leftleaf)}}$ = LeftLeaf Residual / 8

= -1911726.16 / 8 = **-238965.77**

$\text{Prediction}_{\textbf{(Rightleaf)}}$ = RightLeaf Residual / 8

= 1885081.33 / 8 = **235635.17**

We compute the first Decision Tree Prediction (F1):

F1 = GlobalAverage + n*(Leaf Prediction)

Where n = 0.1 (learning rate)

F_{Left} = 1757547.02 + 0.1(-238965.77) = **1733650.44**

F_{Right} = 1757547.02 + 0.1(235635.17) = **1781110.54**

Then compute the New Residual R2 (the remaining error) on both leaves

LEFT LEAF				
ROW ID	ACTUAL SALES	F1 LEFT	ACT SALES – F1 LEFT	(ACT SALES – F1 LEFT)^2
1	1643691	1733650.44	-89959.44	8092699000
2	1641957	1733650.44	-91693.44	8407700000
3	1611968	1733650.44	-121.682.44	14806600000
4	1409728	1733650.44	-323922.44	104925000000
5	1554807	1733650.44	-178843.44	31985800000
6	1439542	1733650.44	-294108.44	86500700000
7	1472516	1733650.44	-261134.44	68191300000
8	1404430	1733650.44	-329220.44	108386000000
TOTAL				**431295800000**

RIGHT LEAF				
ROW ID	ACTUAL SALES	F1 RIGHT	ACT SALES – F1 RIGHT	(ACT SALES – F1 LEFT)^2
9	2136989.46	1781110.54	355878.92	126650000000
10	2137809.50	1781110.54	356698.96	127234000000
11	2124451.54	1781110.54	343341.00	117882000000
12	1865097.27	1781110.54	83986.73	7053770000
13	1991013.13	1781110.54	209902.59	44059100000
14	1990483.78	1781110.54	209373.24	43837000000
15	1946070.88	1781110.54	164960.34	27212000000
16	1750197.81	1781110.54	-30912.73	955590000
TOTAL				**494984000000**

Sum of Squares Total (SST) = $(\sum R_1^2{}_{left}) + (\sum R_1^2{}_{right})$

= 516658000000 + 579195000000 = 1,095,853,000,000

Sum of Squares Errors (SSE) = $(\sum R_2^2{}_{left}) + (\sum R_2^2{}_{right})$

= 431295800000 + 494984000000 = 926,279,800,000

Computing the RMSE and the R^2 Score:

RMSE = $\sqrt{(SSE/N)}$ = $\sqrt{(926279800000/16)}$ = 240,608

R2 = 1-[926,279,800,000/1,095,853,000,000]

= 0.1548 ≈ 15.48%